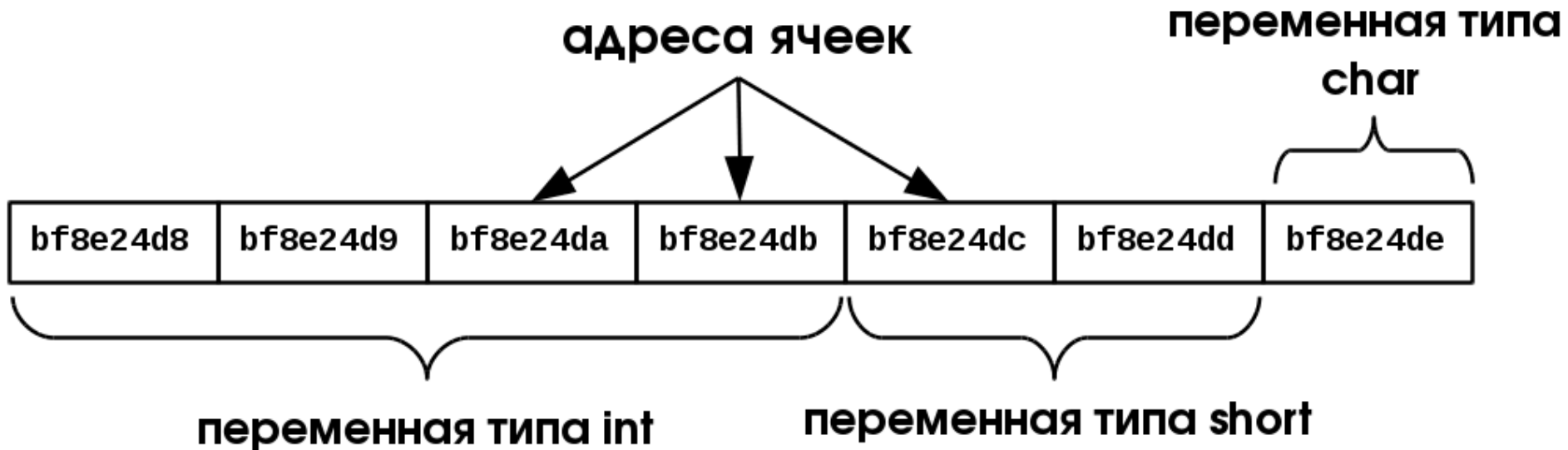


**Адресуемость памяти.
Указатели и ссылки**

Адресуемость памяти

- Номер ячейки памяти называется **адресом**.



- При объявлении переменной с ее именем будет связан адрес байта с наименьшим значением адреса среди тех байтов, которые будут заняты значением этой переменной.
- Для хранения значения переменной отводится такое количество байтов, которое требуется для хранения соответствующего типа данных.

sizeof

- Оператор sizeof возвращает размер в байтах объекта или типа данных.

- Синтаксис:

```
sizeof ( type name );
```

```
sizeof ( object );
```

```
sizeof object;
```

- Применение sizeof к массиву дает **количество байтов**, занимаемых массивом, а не **количество его элементов** и не размер в байтах каждого из них.

Операция «взятия адреса»

- Адрес обычной переменной можно получить с помощью операции & «взятия адреса» (address of).
- `int a;`
- `cout << &a;`
- Адреса в программе представлены в виде шестнадцатеричных чисел с префиксом '0x' в начале, поскольку такое представление (по сравнению с двоичным) выглядит более компактным.

```

#include <iostream>
using namespace std;

int main() {
    char a[]("#");
    cout << a << '\t' << &a << endl;
    short b(1000);
    cout << b << '\t' << &b << endl;
    int c(1000000);
    cout << c << '\t' << &c << endl;
    double d(0.02);
    cout << d << '\t' << &d << endl;
    return 0;
}

```

#	0xbf8e24de
1000	0xbf8e24dc
1000000	0xbf8e24d8
0.02	0xbf8e24d0

Указатели

Указатели (pointer) – это переменные специального типа, значениями которых являются адреса различных объектов программы. Если мы используем имя того или иного объекта для извлечения его значения или для изменения его значения, то принято говорить о непосредственном (прямом) доступе к объекту. В том случае, когда *адрес* объекта помещен в *указатель*, то речь идет о косвенном доступе к объекту, на который «смотрит» *указатель*.

- Идеи такой *косвенной адресации* зародились еще в архитектуре ЭВМ первого поколения, когда *адрес* нужной ячейки памяти помещался в специальный *регистр*.
- *Доступ по содержимому регистра* предоставлял программистам более широкие возможности за счет наличия машинных команд изменения содержимого такого регистра (автоматическое увеличение-инкрементирование или уменьшение-декрементирование на 1) и использование РА в командах организации циклов.

Объявление указателей

- В языках C/C++ различают **три категории указателей**:
- **Первая категория** указателей предназначена для хранения адресов данных определенного типа (по терминологии языка *Паскаль* – типизированные указатели). При их объявлении указывается *тип данных*, на которые эти указатели могут "смотреть".
- **Ко второй категории** относятся указатели, которые могут "смотреть" на данные любого типа. При их объявлении используется *служебное слово* **void**.
- **Третью группу** составляют указатели, значениями которых могут быть только адреса точек входа в функции.

Объявление указателей

- Для объявления одного указателя с именем p1 или нескольких указателей с именами p1, p2, ..., которые должны будут "смотреть" на объекты какого-либо типа, используется одна из следующих синтаксических конструкций:
 - <тип элемента> *p1;
 - <тип элемента> *p1, *p2, ...;
- Объявленные указатели еще никуда конкретно не смотрят – в выделенных им участках памяти находится «мусор».
- И одна из типичных ошибок – попытка записать что-либо по указателям, которым еще не присвоены значения.

Примеры

```
int x=2, y;
```

```
int *p1=&x; //инициализация  
адресом переменной x
```

```
int *p2 (&x); //инициализация  
адресом переменной x
```

```
int *p3=p1; //инициализация  
значением другого указателя
```

```

#include <iostream>
using namespace std;

int main() {
    int d = 10, k = 20;
    int *p; // объявление целого указателя
    // объявление указателя с инициализацией
    int *t = &d;
    p = &d; // присваивание указателю адреса переменной d

    cout << " d => " << d << endl
         << " p => " << p << endl;

    *p = 5; // изменяется значение переменной d
    cout << " d => " << d << endl;

    p = &k; // теперь указатель указывает на переменную k
    cout << " p => " << p << endl;

    d = *p; // d присваивается значение переменной k
    cout << "*p => " << d << endl;

    p = t; // Присваивание значения одного указателя другому
    cout << " p => " << p << endl
         << " // Указатель тоже имеет адрес:
         << "&p => " << &p << endl;
    return 0;
}

```

```

d => 10
p => 0xbfe59b58
d => 5
p => 0xbfe59b54
*p => 20
p => 0xbfe59b58
&p => 0xbfe59b50

```

Указатель `p` хранит адрес переменной `d` (или указывает на переменную `d`), но, как переменная, имеет свой адрес

Имя переменной	Значение	Адрес
d	10	bfe59b58
k	20	bfe59b54
p	bfe59b58	bfe59b50

как получить значение и адрес переменной с помощью операций "*" и "&"

Значение	Адрес
d	&d
*p	p

Операции с указателями

- **разыменование (*)** - позволяет получить доступ к значению находящемуся по адресу, который сохранен в указателе.
- **присваивание**
- **арифметические операции** (сложение с константой, вычитание, инкремент ++, декремент --)
- **присваивание значения nullptr** (нулевой указатель). Например, `int *p = nullptr;`

```

#include <iostream>
using namespace std;

int main() {
    int a = 5, b = 10, c = 15, d = 20;
    int *p;
    cout << &a << "\n"
         << &b << "\n"
         << &c << "\n"
         << &d << "\n" << endl;
    p = &d;
    // получаем значения переменных
    // разыменованием указателя
    // с последующей его инкрементацией
    cout << *p++ << "\n"
         << *p++ << "\n"
         << *p++ << "\n"
         << *p++ << endl;
    // еще одно обращение к данным приведет
    // к неопределенным результатам!
    cout << *p << endl;
    return 0;
}

```

```

0xbfd64cf8
0xbfd64cf4
0xbfd64cf0
0xbfd64cec

```

```

5
10
15
20
-1076474628

```


Ссылки

- **Ссылки (reference)**, как и указатели, являются реализацией ссылочного типа.
- Для описания ссылки используется операция **&**.
- Ссылка должна быть проинициализирована в момент описания именем переменной, на которую она будет ссылаться.

```
int a = 10;
```

```
int &b = a;
```

- При описании ссылки происходит не инициализация копированием (как в случае с переменной **a**), а **связывание** её с объектом (т. е. с этой переменной).
- Таким образом, **ссылка является псевдонимом объекта.**
- В этом ключе некорректно именовать ссылку - *ссылочной переменной*.
- При обращении к ссылке происходит обращение к значению того объекта, с которым связана данная ссылка.

Правила использования ссылок в программе:

- Поскольку ссылка связывается с объектом она должна **быть проинициализирована в момент описания.**
- После инициализации ссылка **не может быть** связана с иным объектом.
- Поскольку ссылки - это не объекты, то **нельзя определять ссылки на ссылки.**
- Тип ссылки должен совпадать с типом объекта на который она ссылается.
- **Нельзя определять указатель на ссылки.**

Связь указателей и ссылок

- Ссылка — это, по существу, указатель, который жестко привязан к области памяти, на которую он указывает, и который автоматически разыменовывается, когда происходит обращение по имени ссылки. Ссылка, в отличие от указателей, не занимает места в памяти.
- Связь между указателями и ссылками можно установить следующим образом: операция взятия адреса (&) получает указатель, ссылающийся на тот же самый объект при переходе по ссылке, а ссылка, которая инициализирована при разыменовании (*) указателя, будет указывать на тот же объект, что и указатель.

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 20;
    cout << "a => " << a << endl;
    cout << "b => " << b << endl;
    int *p = &a;
    int &c = b; // связывание ссылки
    cout << "c => " << c << endl;
    c = *p; // присваивание значения разыменовывания
           // объекта на который ссылается ссылка
    cout << "c => " << c << endl;
    c = 20; // присваивание объекту с которым связана ссылка
    cout << "c => " << c << endl;
    return 0;
}
```

a => 10

b => 20

c => 20

c => 10

c => 20

- **Важно отличать операцию взятия адреса от ссылки.**
- Операция взятия адреса используется для уже созданного объекта с целью получения его адреса, а ссылка - это только задание альтернативного имени объекта.
- Ссылки, в отличие от указателей, предоставляют более компактное, понятное, безопасное и удобное средство работы с данными. Поэтому, использование ссылок (вместо указателей) является хорошим тоном в программировании на C++.

Использование указателей:

функции с переменным числом параметров

- В C++ можно создавать функции с переменным числом параметров.
- Параметры, их количество и типы, становятся известны только во время вызова функции.
- Формат описания функций с переменным числом параметров:

```
тип имя (<список явных параметров>, ...)  
{  
<тело функции>  
}
```

- После списка явных параметров следует необязательная запятая и **троеточие**, которое сообщает компилятору, что контроль типов и количества параметров при вызове функции не следует.
- При создании функций с переменным числом параметров необходимо предусмотреть способ определения количества параметров и их типов.
- Используется для этих целей два способа:
 1. один из параметров определяет число параметров функции;
 2. в списке явных параметров задается параметр, указывающий на конец списка параметров.

- Если не один из этих способов не используется, тогда можно использовать ***специальный набор макроопределений***.
- Параметры функции помещаются в стек, при этом первый параметр оказывается в вершине стека.
- Переход от одного параметра к другому осуществляется **с помощью указателей**.
- Для использования в программе функций с переменным числом параметров, при вызове которых можно использовать параметры разных типов, удобно использовать специальный набор макроопределений (заголовочный файл **`stdarg.h`**).

Пример

```
#include <stdio.h>
#include <conio.h>
#include <iostream>
void main()
{
    int sum(int n,...); //прототип функции с переменным
                        //числом параметров
    cout << "\n 4+6=" << sum(2,4, 6);
    cout << "\n 1+2+3+4+5+6=" << sum(6,1,2,3,4,5,6);
    cout << "\n Parametrov net.Summa ravna:" << sum(0);
    getch();
}

int sum(int n,...)//n-число суммируемых параметров
{
    int s=0;
    int *p = &n; //получение адреса параметров в стеке
    for(int i=1; i<=n; i++)
        s+=*++p; //суммируем числа
    return s;
}
```